Expressing a Rational Function as a Product of Given Letters

(the method behind `SolveFactorPowers`)

TriangleHeptagon3 – `4DBasis` notes

November 10, 2026

1 The problem

Given a target rational function T and a list of candidate factors ("letters") $f_1, \ldots, f_k$, find integer exponents $n_1, \ldots, n_k$ such that

$$T \;=\; \text{const} \cdot \prod_{i=1}^{k} f_i^{\,n_i}.$$

This is exactly the bookkeeping needed when building a symbol alphabet for a dlog form [5, 6]: each differential you reconstruct factors into some combination of the "letters" already found, and you need the precise combination, not a guess from eye-balling exponents on expressions that can run to dozens of terms. This document is not a trial-and-error recipe – it is four exact, checkable steps, each with a specific failure mode that was actually hit and fixed while building `FactorAlphabetSolver.wl`.

2 Step 1 – turn a multiplicative problem into a linear one

Every rational function factors uniquely (up to order and sign) into irreducible polynomials:

$$f_i = c_i \prod_p p^{\,a_{ip}}, \qquad T = c_T \prod_p p^{\,a_{Tp}},$$

where p ranges over irreducible polynomials and `FactorList` supplies the exponents a_{ip}, a_{Tp} directly. The equation $T = \prod f_i^{n_i}$ is then, irreducible-by-irreducible,

$$\sum_{i=1}^{k} n_i \, a_{ip} \;=\; a_{Tp} \qquad \text{for every irreducible } p \text{ appearing anywhere.}$$

This is an ordinary linear system in the unknowns n_i – one equation per distinct irreducible factor, ordinarily *far* more equations than unknowns, which is exactly what makes the method work: an accidental or approximate match is astronomically unlikely once there are a dozen or more irreducible factors pinning down the n_i.

Implementation. Build the exponent vector of every candidate and of the target against the same ordered list of irreducibles, forming a matrix M (candidates $\times$ irreducibles) and a target vector v; the system is $n \cdot M = v$.

3 Subtlety 1 – sign is not canonical

`FactorList` does not guarantee the same sign for "the same" irreducible factor when it is extracted from two differently-built expressions: one expression may record a factor as F, another as $-F$, with the overall numeric prefactor silently compensating. A naive key-by-key match on the raw factor then sees *zero* overlap between two expressions that actually share that factor, and the linear system reports no solution even though one exists – this is exactly what happened on the second worked example in this project (a polynomial denoted `f7` came back as its own negative inside a different target expression).

Fix. Canonicalize every factor's sign before using it as a dictionary key:

```
CanonicalFactor[f_] := Module[{nf = Expand[-f]},
    If[OrderedQ[{f, nf}], f, nf]];
```

Since OrderedQ$[\{F, -F\}]$ and OrderedQ$[\{-F, F\}]$ are logical complements of each other, this always resolves to the *same* canonical representative regardless of which sign you started from – so F and $-F$ now collide to one key, and the true overall sign is recovered separately at the end via a direct numeric check ($T/\prod f_i^{n_i}$ must come out to a plain constant).

4 Subtlety 2 – the letters are usually not independent

Solving $n \cdot M = v$ over the integers does not generally give a unique n. In the first worked example here, three of the candidates were

$$f_1 = c_1^2 G, \qquad f_2 = c_1(c_1 - c_2)G, \qquad f_3 = (c_1 - c_2)^2 G$$

for a common polynomial G, and these satisfy the identity

$$f_2^2 = f_1 f_3$$

identically – an unsurprising relation once noticed (they are literally $c_1^2, c_1(c_1 - c_2), (c_1 - c_2)^2$ times the same G, a perfect-square pattern), but easy to miss by eye in a page of symbol output. Each such multiplicative relation among the candidates ("syzygy") is a vector in the integer null space of M^T, and contributes one free integer parameter to the general solution: the decomposition is a whole *lattice* of equally valid answers, not one.

`Solve[..., Integers]` already returns exactly this structure – a particular solution plus free parameters `C[1], C[2], ...` – so detecting non-uniqueness is immediate (`isUnique -> False`) and costs nothing extra.

5 Subtlety 3 – do not enumerate the lattice

The first implementation of this tool picked a "simplest" representative by scanning all small integer values of the free parameters:

```
Do[ (* check candidate exponent vector, keep the best *)
 ..., {pt, Tuples[Range[-6,6], numFreeParams]}]
```

This works fine when there is one free parameter (13 combinations to check), but the number of combinations is 13^m for m free parameters – and real symbol alphabets routinely produce a dozen

or more independent syzygies among the candidates. In actual use this hit $m = 15$, i.e. `Tuples` was asked to build $13^{15} \approx 5 \times 10^{16}$ tuples, which exhausts available memory before producing anything. Replacing the scan with `Minimize` (an exact integer linear program, minimizing $\sum |n_i|$ subject to the same linear constraints) avoids building all combinations explicitly, but still did not finish within 60 s on a 20-candidate, 15-free-parameter test – the branch-and-bound search underneath still scales badly on this many integer variables with an ℓ_1 objective.

The actual fix: lattice reduction, not search. The set of valid solutions is $p_0 + \mathrm{span}_{\mathbb{Z}}(b_1, \ldots, b_m)$ for a particular solution p_0 and an integer basis $b_1, \ldots, b_m$ of the null space (read off directly from `Solve`'s output by setting each free parameter to a unit vector in turn). Finding the *shortest* (or just a short, sensible) representative is then the classical **closest vector problem** [4]: find the lattice point nearest to $-p_0$. This has a standard, polynomial-time, non-enumerative solution:

1. **LLL-reduce** the basis: `reducedBasis = LatticeReduce[{b_1,...,b_m}]`. The Lenstra–Lenstra–Lovász algorithm [1] replaces an arbitrary (possibly very skewed) basis with one that is short and nearly orthogonal, *without changing the lattice it spans* (see also [3] for a computational treatment).

2. **Babai's rounding** [2]: project $-p_0$ onto the span of the reduced basis by least squares, $c \approx \mathrm{LeastSquares}\big(\mathrm{reducedBasis}^T, -p_0\big)$, then round c to the nearest integer vector.

3. The answer is $p_0 + \mathrm{Round}(c) \cdot \mathrm{reducedBasis}$ – guaranteed to still solve the original linear system exactly (adding any integer combination of null-space vectors to a particular solution gives another particular solution), and in practice very short because the basis was reduced first. Babai's algorithm is only guaranteed near-optimal rather than exactly optimal in general, but for this purpose "short" is all that is needed – there is no physical meaning attached to the exponent vector being the unique minimal one, only to it being small and correct.

Crucially, none of these three steps involves any combinatorial enumeration: each is a fixed, polynomial-time linear-algebra computation regardless of how many free parameters m turns out to be.

6 Verification

Every step above was checked, not assumed:

- **Correctness on the original worked examples.** Both hand-derived target expressions from this project reproduce the expected exponents exactly, with the final numeric check $T / \prod f_i^{n_i}$ coming back exactly 1 in both cases (not just "a solution exists" – the actual target is reproduced).

- **The failure mode, reproduced on purpose.** A synthetic test with 20 monomial candidates built from only 5 underlying symbols (so the null space has dimension $20 - 5 = 15$, matching the real failure) was constructed with a known target. The `Tuples`-scanning version could not even be attempted at this size (it is exactly what failed in practice); the `Minimize`-based version exceeded a 60 s timeout; the lattice-reduction version solved it in 0.01 s, with $T / \prod f_i^{n_i} = 1$ confirmed.

- **Scaling further.** A second synthetic test with 40 candidates from 6 underlying symbols (null-space dimension 34) solved correctly in 0.08 s – confirming the method's cost does not grow combinatorially with the number of relations among the letters.

7 Summary

Step	What it does	Why it is needed
Factor into irreducibles	Turns $\prod f_i^{n_i} = T$ into a linear system	Exponents add; products do not
Canonicalize factor sign	Makes F and $-F$ the same key	`FactorList`'s sign is not canonical
`Solve[...,Integers]`	Particular solution + free-parameter basis	Exposes non-uniqueness directly
`LatticeReduce` (LLL)	Short, near-orthogonal null-space basis	Makes rounding meaningful
Least squares + `Round`	Babai's algorithm: a short representative	Polynomial time, no enumeration

The result, `SolveFactorPowers[target, candidates, names]` in `4DBasis/FactorAlphabetSolver.wl`, is exact (verified against known answers), flags non-uniqueness explicitly rather than hiding it, and its cost is governed by linear algebra, not by how many ways the candidates happen to overlap.

References

[1] A. K. Lenstra, H. W. Lenstra Jr., L. Lovász, *Factoring polynomials with rational coefficients*, Mathematische Annalen **261** (1982), 515–534.

[2] L. Babai, *On Lovász' lattice reduction and the nearest lattice point problem*, Combinatorica **6** (1986), 1–13.

[3] H. Cohen, *A Course in Computational Algebraic Number Theory*, Graduate Texts in Mathematics **138**, Springer, 1993.

[4] D. Micciancio, S. Goldwasser, *Complexity of Lattice Problems: A Cryptographic Perspective*, Kluwer Academic Publishers, 2002.

[5] A. B. Goncharov, M. Spradlin, C. Vergu, A. Volovich, *Classical Polylogarithms for Amplitudes and Wilson Loops*, Phys. Rev. Lett. **105** (2010) 151605, arXiv:1006.5703.

[6] C. Duhr, *Mathematical aspects of scattering amplitudes*, TASI 2014 lecture notes, arXiv:1411.7538.